JVM – Simulator

Student Name: Rakshith B
Program: 3rd Semester, B.Tech Computer Science and Engineering
Institution: Chanakya University

1. Project Description
This project implements a custom Java Virtual Machine (JVM) simulator built in Python. It executes a specific subset of Java bytecode instructions using a stack-based architecture. The simulator enforces strict memory constraints, capping both the operand stack and the local variable register set at a maximum of 10 elements each.

2. Design and Architecture
The simulator reads a bytecode program from a standard text file, processing one instruction per line.

- **Execution Logic:** A standard fetch-decode-execute loop is managed by a run_pro() method.
- **Program Counter (PC):** Execution flow is tracked using a zero-indexed PC. Branching instructions evaluate the top of the stack and modify the PC directly to jump to the target line index.
- **Mathematical Operations:** Arithmetic relies strictly on LIFO (Last-In-First-Out) principles. The right-hand operand is popped first, followed by the left-hand operand, to guarantee accurate subtraction and integer division.
- **Error Handling:** The simulator includes active checks for OverflowError (exceeding the 10-element stack limit), IndexError (stack underflow), and ZeroDivisionError.

3. Bytecode Test Cases
Three bytecode programs validate the simulator's logic:

- **Task A (Min/Max):** Reads three integers, compares them using conditional branching (iflt, ifgt) to evaluate the minimum and maximum values, and prints both results.
- **Task B (Sorting):** Reads three integers and sorts them in ascending order using an unrolled comparison and swapping method, before printing the sequence.
- **Task C (Matrix Addition):** Sequentially reads the elements of two 2×2 matrices, computes their positional sum, and prints the resulting matrix in the original order.
-

4. How to Run

1. Ensure the Python script (simulator.py) and the test case text files (task_a.txt, task_b.txt, task_c.txt) are in the same directory.
2. Modify the file path in the sim.load_program() method call at the bottom of the script to select the desired test case.
3. Execute the script via the terminal.